

Autonomous Parking and Line Following via Computer Vision

6.4200: Team 5: Lab 4

March 16, 2026

Kelsey Fontenot (2), Yifan Kang (6-4),
Dante Suazo (6-4), He'yun Zhi (6-3 & 21L)

This lab's goals were focused on computer vision (CV) application and integration into physical systems.

For this class:

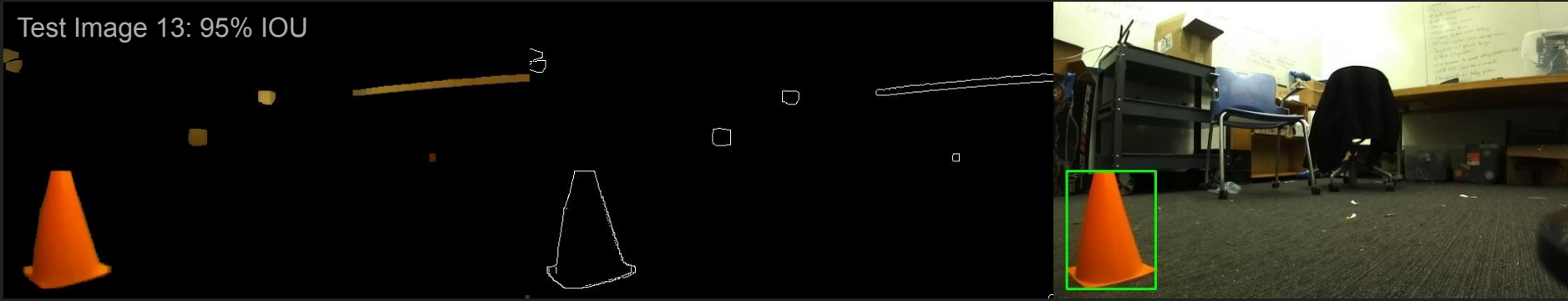
- Applied CV algorithms and tactics to recognize an orange cone
- Controlled robot to change its trajectory in real time given visual feedback
- Use combined knowledge across modules to build an (orange) line follower

In larger context:

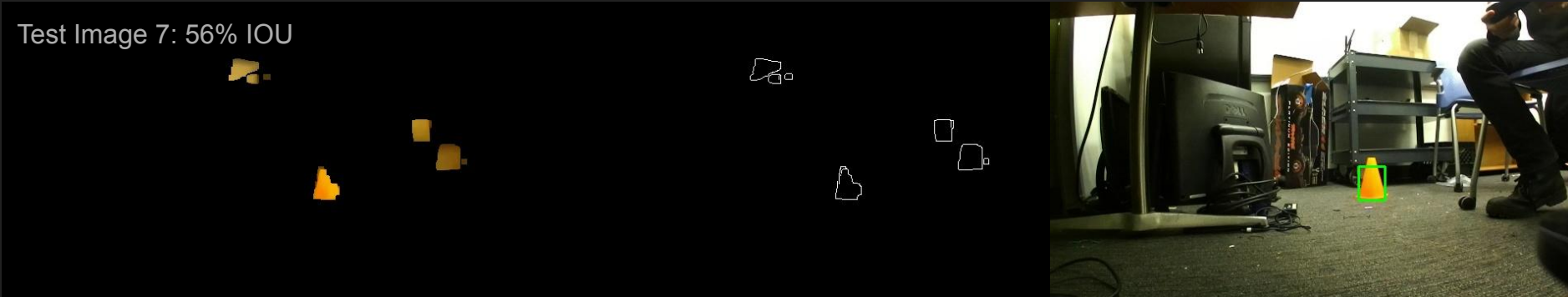
- CV enables safer, more accurate perception, allowing vehicles to understand their surroundings to make driving decisions, such as reacting to obstacles.

Color segmentation correctly identifies cone position for 95% of provided test cases.

Test Image 13: 95% IOU



Test Image 7: 56% IOU

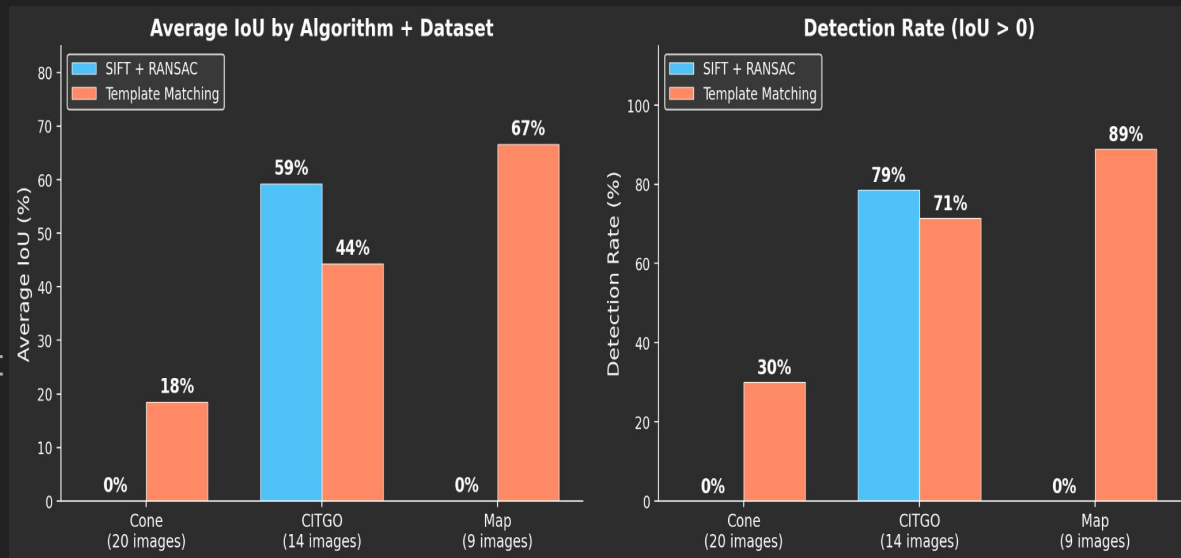


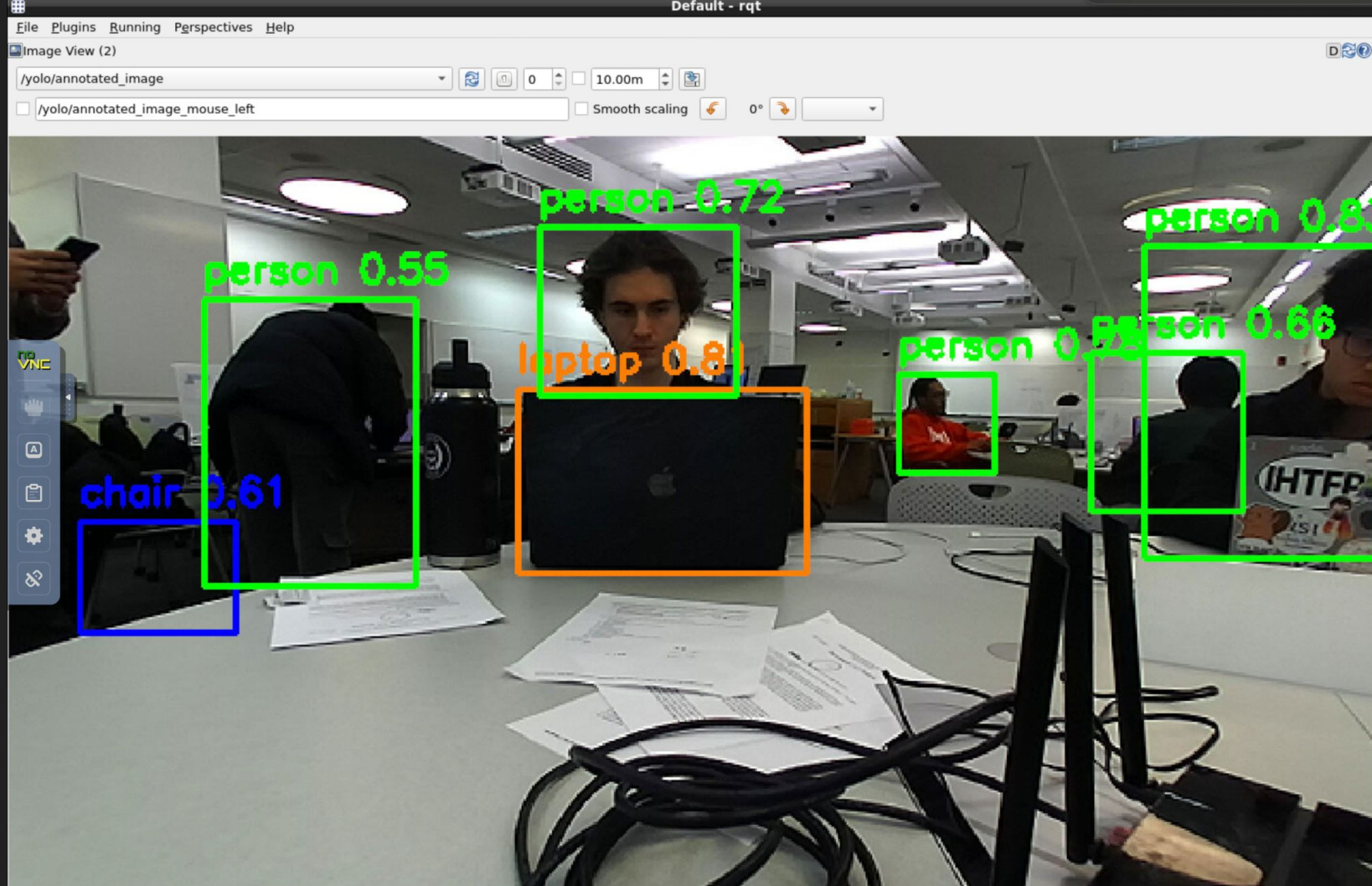
SIFT detects feature-rich targets; template matching generalizes to structural ones -- neither solves textureless objects like the cone.

SIFT dominates CITGO -- distinctive keypoints handle viewpoint changes

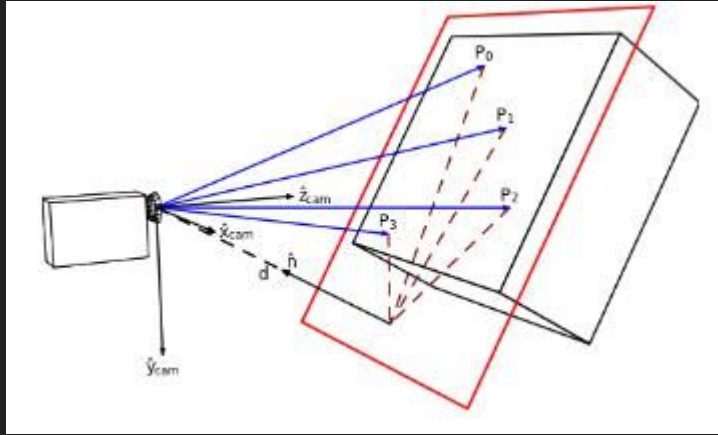
Template matching dominates map -- edge structure preserved in 2D

Neither solves the cone -- its strongest feature is color, not shape or texture





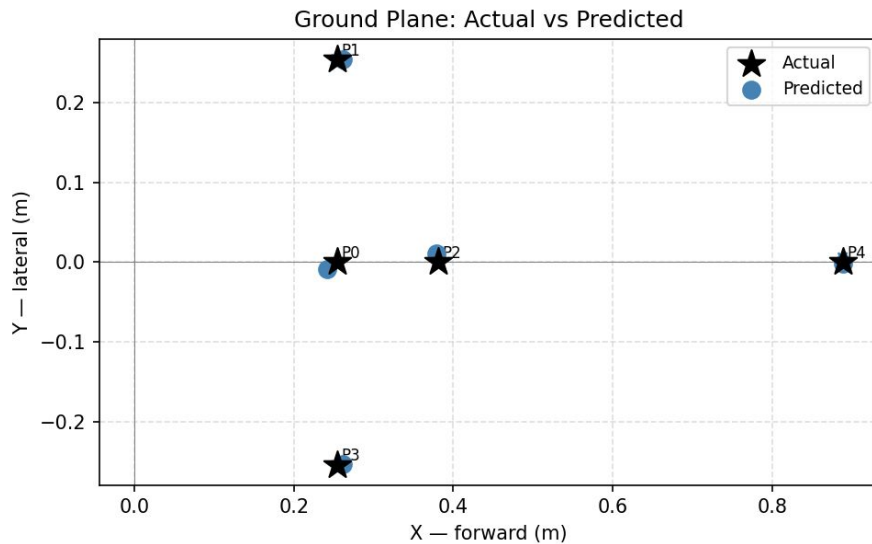
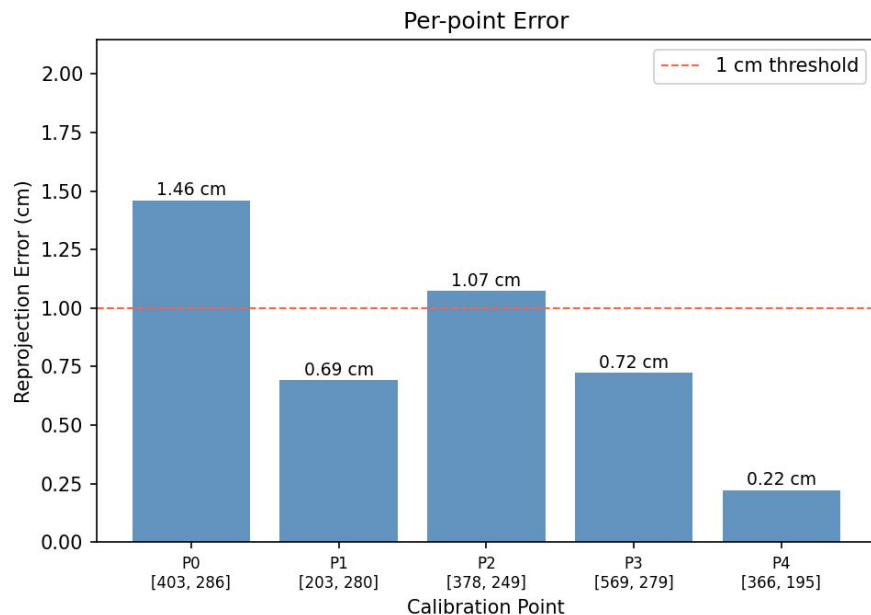
Homography Transformation



$$s \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

Analysis

Homography Reprojection Error

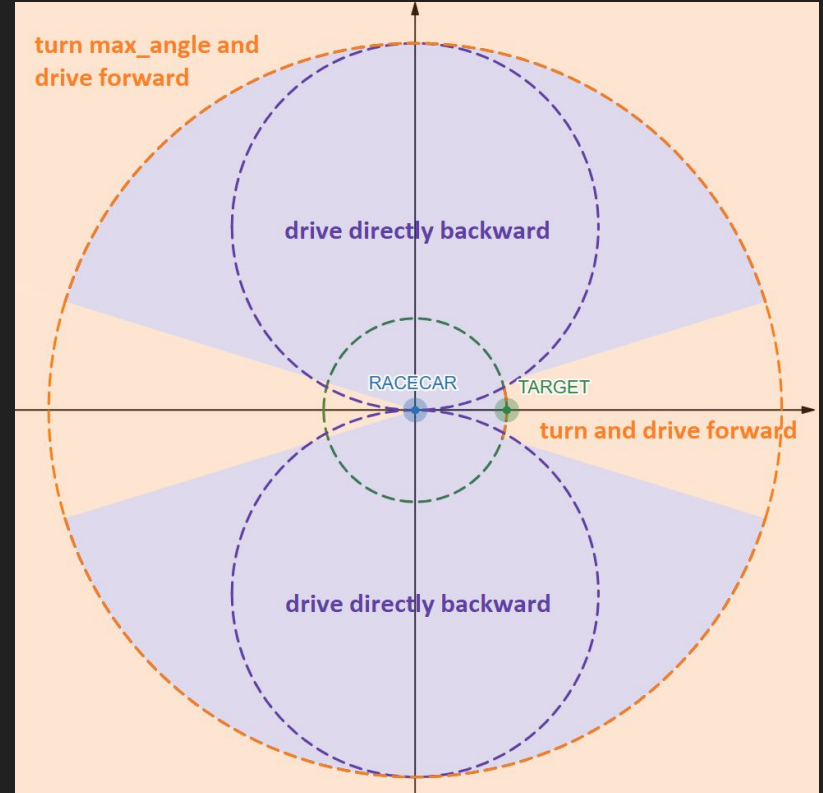


We calculated how to park a set distance directly in front of any given target coordinates.

Diagram of what the car is told to do for each potential cone location relative to itself.

Turns toward target (as far as possible) and drives forward if target is in orange;

Drives directly backward if target is in purple.



Our design choices for our parking controller prioritize practicality, simplicity, and completeness.

Modifiable “acceptable distance error”,
“acceptable angle error”, “max steering angle”.

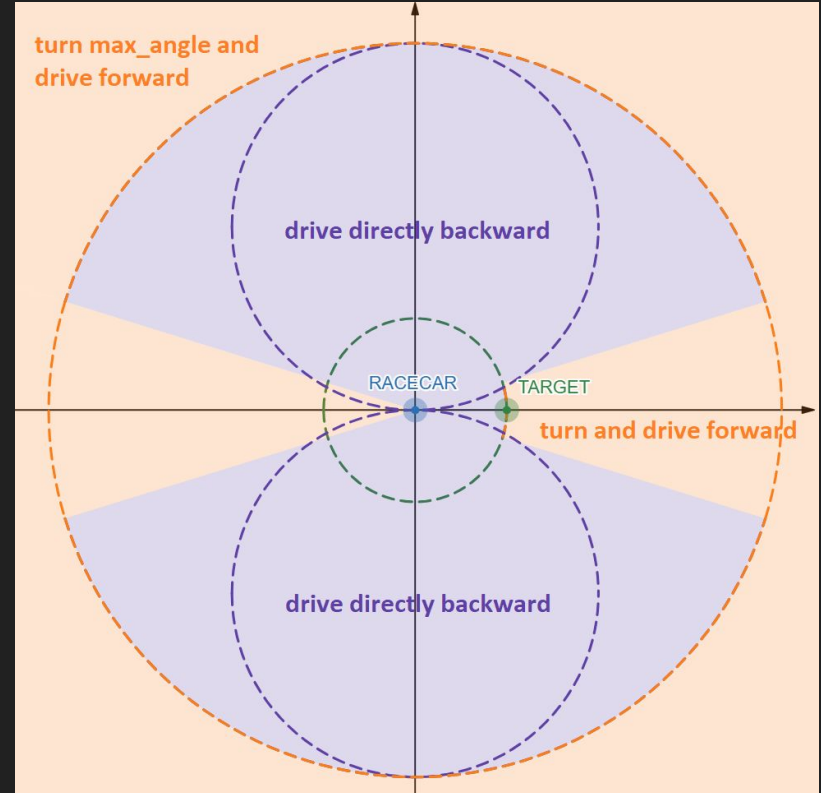
Slows down while approaching target.

Drives backward as little as possible.

Keeps target within sight.

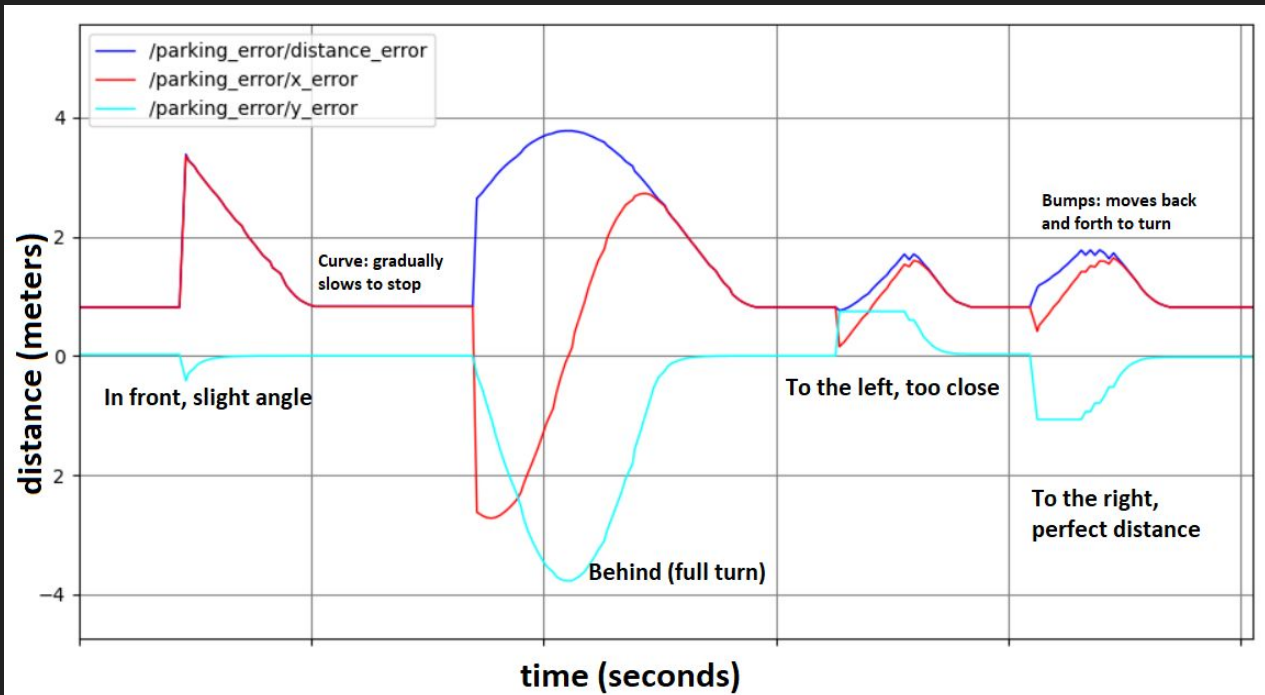
Approximates twin circle “unreachable zones”
into twin sectors.

All possibilities handled, no matter how
unrealistic.



Our simulated parking controller performs smoothly and efficiently in any scenario.

Error measurements between the car and target for four representative test cases:



We put everything together on our physical robot to make a successful real-life autonomous parking controller.



Our line follower expands on our parking controller with only a few small changes.



We still want to improve our line follower for robustness.

We want our racecar to be able to handle dashed and incomplete lines as well as go through corners more accurately.



Lessons Learned

Kelsey: Need to build and troubleshoot segmentation mask across multiple test cases, instead of just one.

Yifan:

Dante: Selecting a vision algorithm based on target properties (texture, color, structure) matters more than algorithm complexity -- SIFT is wasted on a textureless cone.

He'yun: Compartmentalizing a large engineering project into standalone modules with tailored testing and evaluation for each is a necessary fundamental skill

Thank you!

